

计算机程序设计基础(2)

--- C++程序设计(7)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 62796193/13901216180

2020. 3.

1

5.9 动态束定与虚函数

- virtual所修饰的成员函数，被称为虚函数。
- 延迟到程序运行时进行的束定称为动态束定（动态绑定）。
- 当一个类是从一个或多个类派生出来时，往往会出现多个函数名相同且参数个数和参数类型也完全相同的成员函数，分别实现不同的功能。但这些函数不是重载函数（重载函数必须参数个数或参数类型不同，可以静态束定），编译器无法在编译时静态束定到底是要调用哪一个成员函数，即使可以静态束定，往往也不一定符合编程者的真正意愿而导致结果错误。

2

- 虚函数延迟到程序运行时再行动态束定，由this指针决定调用（通过虚函数表(vtable)）哪一个同名的成员函数。
- 需要在重名的成员函数前加上 virtual 声明。
- 包含虚函数的类被称为多态类。
- 多态性（polymorphism）是面向对象的重要特征之一。
- 面向对象的另外两个重要特征是：封装、继承。
- 提示：**面向对象的三个重要特征是：封装、继承、多态。

3

例如：

```
#include <iostream>
using namespace std;
class Shape { // 点类(X,Y)
public: Shape(double x=0, double y=0);
       double Area( ) const;
private: double X,Y;
};
Shape::Shape(double x, double y):X(x),Y(y){ }
double Shape::Area( ) const
{ return 0.0; }
class Circle : public Shape { // 圆类=点类+半径
public: Circle(double x, double y, double r);
       double Area( ) const;
private: double Radius;
};
```

4

```
Circle::Circle(double x, double y, double r)
: Shape(x,y), Radius(r) { } // Shape(x,y)调用基类构造函数
double Circle::Area( ) const
{ return 3.14159 * Radius * Radius; }
void fun(const Shape &sp) // 常引用对象调用的必须是常函数
{ cout << sp.Area( ) << endl; } // 去掉Area( )后的const将编译错误
void main( )
{ Circle C(2.0, 5.0, 3.65637);
  Shape S(3.0, 4.0);
  Shape *p[2]={&S, &C};
  fun(C); fun(S);
  for (int i=0; i<2; i++)
    cout << p[i]->Area( ) << endl;
}
```

类型兼容性规则：基类指针可以指向它的派生类对象，反之错误。

5

如果把程序修改为：

```
#include <iostream>
using namespace std;
class Shape { // 点类(X,Y)
public: Shape(double x, double y);
       virtual double Area( ) const; // 加上virtual
private: double X,Y;
};
Shape::Shape(double x, double y):X(x),Y(y){ }
double Shape::Area( ) const
{ return 0.0; }
class Circle : public Shape { // 圆类=点类+半径
public: Circle(double x, double y, double r);
       virtual double Area( ) const; // 加上virtual
private: double Radius;
};
```

6

```

Circle::Circle(double x, double y, double r)
: Shape(x,y), Radius(r)
{ }
double Circle::Area( ) const
{ return 3.14159 * Radius * Radius; }
void fun(const Shape &sp)
{ cout << sp.Area( ) << endl; }
void main( )
{ Circle C(2.0, 5.0, 3.65637);
  Shape S(3.0, 4.0);
  Shape *p[2]={&S, &C};
  fun(C); fun(S);
  for (int i=0; i<2; i++)
    cout << p[i]->Area( ) << endl;
}

```

运行结果:

```

42
0
0
42

```

7

5.9 动态绑定与虚函数 (续1)

在派生类中重定义虚函数时，必须满足：

1. 与基类的虚函数参数个数相同；
2. 与基类的虚函数参数一一对应；
3. 函数返回类型或修饰词（如const）与基类的虚函数相同，或都返回指针或引用(可以不同类型)。

若满足上述三条件，有时可以省略virtual关键字，编译器会自动认为是同族虚函数。但从程序可读性角度考虑，建议保留virtual关键字。

8

5.10 纯虚函数

- ◆ 纯虚函数是抽象类中的虚函数，在函数名后直接赋值为0，无函数体。
- ◆ 纯虚函数不同于空函数，是抽象的。
- ◆ 纯虚函数为抽象类中的成员函数。
- ◆ 包含纯虚函数的类被称为抽象类。
- ◆ 抽象类（无论类中是否有数据成员）无法实例化，因此不能用抽象类定义对象。
- ◆ 纯虚函数声明格式：

virtual 函数类型 函数名(参数表)=0;

9

```

#include <iostream>
#include <cmath>
using namespace std;
class Figure{ // Figure是一个抽象类
public:
    virtual double getArea( )=0;
}; // 纯虚函数，为整个类族提供了通用的外部接口语义
class Rectangle: public Figure {
public:
    Rectangle(double height=0,double width=0)
    { this->height=height;
      this->width=width;
    }
    double getArea( ) { return height*width; }
protected:
    double height, width;
};

```

10

```

class Triangle : public Figure {
public:
    Triangle(double a=0, double b=0, double c=0)
    { this->a=a; this->b=b; this->c=c; }
    double getArea( )
    { double s=(a+b+c)/2;
      return sqrt( s*(s-a)*(s-b)*(s-c) );
    }
protected:
    double a, b, c;
};
void main( )
{ Figure *Fig[ ]={new Rectangle(3,4), new Triangle(3,4,5)};
  // Figure基类指针可以指向其任一个派生对象
  cout << Fig[0]->getArea( ) << ' ' << Fig[1]->getArea( );
  delete Fig[0]; delete Fig[1];
}

```

这里可以定义Figure指针，但不能定义Figure对象，因为抽象类Figure不能实例化。

11

运行结果:
12 6
请按任意键继续...

如果把主函数改为:

```

void main( )
{ Figure *Fig[ ]= {new Figure, new Rectangle(3,4), new Triangle(3,4,5)};
  cout << Fig[0]->getArea( ) << ' ' << Fig[1]->getArea( );
  cout << ' ' << Fig[2]->getArea( ) << endl;
  delete Fig[0]; delete Fig[1]; delete Fig[2];
}

```

编译结果:

```

>test.cpp
1> test.cpp(28) : error C2259: "Figure": 不能实例化抽象类
1> 由于下列成员:
1> "double Figure::getArea(void)": 是抽象的
1> test.cpp(6) : 参见"Figure::getArea"的声明
1>test - 1 个错误, 0 个警告

```

12

5.11 虚析构函数

```
#include <iostream>
#include <cmath>
using namespace std;
class Figure{
public:
    virtual double getArea( )=0;
};
class Rectangle: public Figure {
public:
    Rectangle(double height=0,double width=0)
    { this->height=height; this->width=width; }
    ~Rectangle( ) // 加入析构函数, 仅打印信息
    { cout << "destruct Rectangle!\n"; }
    double getArea( )
    { return height*width; }
protected:
    double height, width;
};
```

13

```
class Triangle: public Figure {
public:
    Triangle(double a=0,double b=0, double c=0)
    { this->a=a; this->b=b; this->c=c; }
    ~Triangle( ) // 加入析构函数, 仅打印信息
    { cout << "destruct Triangle!\n"; }
    double getArea( )
    { double s=(a+b+c)/2;
      return sqrt( s*(s-a)*(s-b)*(s-c) );
    }
protected:
    double a, b, c;
};
void main( )
{ Figure *Fig[ ] = {new Rectangle(3,4), new Triangle(3,4,5)};
  for (int i=0; i<2; i++)
    cout << Fig[i]->getArea( ) << endl;
  for (i=0; i<2; i++)
    delete Fig[i];
}
```

运行结果:

12
6
请按任意键继续...
未调用Rectangle和Triangle的析构函数, 因而没有正常释放Rectangle和Triangle对象的动态内存。

14

若在Figure类中加入: virtual ~Figure(){ }

其余不变:

```
#include <iostream>
#include <cmath>
using namespace std;
class Figure{
public:
    virtual double getArea( )=0;
    virtual ~Figure( ){ } // 虚析构函数
};
class Rectangle: public Figure {
public:
    Rectangle(double height=0,double width=0)
    { this->height=height; this->width=width; }
    virtual ~Rectangle( ) // 函数名前面可以加virtual, 也可以不加
    { cout << "destruct Rectangle!\n"; }
    double getArea( )
    { return height*width; }
protected:
    double height, width;
};
```

15

```
class Triangle: public Figure {
public:
    Triangle(double a=0,double b=0, double c=0)
    { this->a=a; this->b=b; this->c=c; }
    virtual ~Triangle( ) // 函数名前面可以加virtual, 也可以不加
    { cout << "destruct Triangle!\n"; }
    double getArea( )
    { double s=(a+b+c)/2;
      return sqrt( s*(s-a)*(s-b)*(s-c) );
    }
protected:
    double a, b, c;
};
void main( )
{ Figure *Fig[ ] = {new Rectangle(3,4), new Triangle(3,4,5)};
  for (int i=0; i<2; i++)
    cout << Fig[i]->getArea( ) << endl;
  for (int i=0; i<2; i++)
    delete Fig[i];
}
```

运行结果:

12
6
destruct Rectangle!
destruct Triangle!
请按任意键继续...

16

- 在Figure类中加入: virtual ~Figure(){ }
- 将 ~Figure()
~Rectangle()
~Triangle()
- 变为同族的**虚析构函数**。
- **注意:** 有虚析构函数, 但没有对应的虚构造函数。
- 虚析构函数使系统在执行过程中进行析构时, 根据this指针所指的当前对象, 动态决定使用这个对象的析构函数进行正确析构。
- 若不是虚析构函数, 编译时将直接静态束定, 将使得delete Fig[i]都使用~Figure(), 不能正确析构。

提示:

虚析构函数是C++中唯一的函数名不同的虚函数族。

17

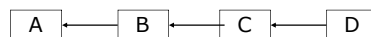
5.12 多继承

- ✓ 前面讨论的是**单继承**: 一个派生类是从一个基类派生而来的。
- ✓ 若一个派生类是同时从两个和两个以上基类派生而来的, 则称为**多继承** (Multiple Inheritance, 多重继承)。

```
class D : public A, protected B, private C
{ ..... }
```

- ✓ 多继承有几种情况:

- 1, B从A派生, C从B派生, D从C派生, 多层(重)派生



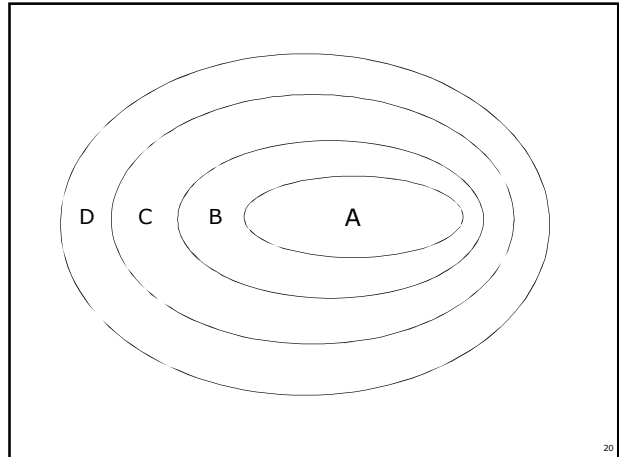
18

```

class A
{ public:  A(...) {...}          // 构造函数
          A(const A &p){...} // 复制构造函数
  .....
};
class B : public A
{ public:  B(...) : A(...) {...} // 构造函数
          B(const B &p) : A(p) {...} // 复制构造函数
  .....
};
class C : public B
{ public:  C(...) : B(...) {...} // 构造函数
          C(const C &p) : B(p) {...} // 复制构造函数
  .....
};
class D : public C
{ public:  D(...) : C(...) {...} // 构造函数
          D(const D &p) : C(p) {...} // 复制构造函数
  .....
};

```

19



20

注意：无论A、B、C类的成员是private还是protected，派生类的构造函数和复制构造函数只能使用其直接继承的基类的构造函数和复制构造函数构造基类对象。

例如：D的构造函数只能使用C的构造函数，而不能直接使用B和A的构造函数构造；只能间接由C的构造函数使用B的构造函数，再间接由B的构造函数使用A的构造函数，从而依次构造出A、B、C基类对象，最终构造出D对象。

- ◆ 写成：

```

class D : public C
{ public: D(...) : A(...),B(...),C(...) {...} // 构造函数
          D(const D &p) : A(p),B(p),C(p) {...} // 复制构造函数
  .....
};

```

 是错误的。
- ◆ 派生类D的析构函数析构时，首先析构D，再调用C的析构函数，C的析构函数先析构C，再调用B的析构函数，B的析构函数先析构B，再调用A的析构函数析构A，至此整个D被析构完毕。

21

多层（重）派生示例：

```

#include <iostream>
using namespace std;
class A
{ public:
  A(int x) { a=x; cout << a << " A构造函数被调用\n"; }
  A(const A &p): a(p.a){ cout << a << " A复制构造函数被调用\n"; }
  ~A() { cout << a << " A析构函数被调用\n"; }
protected:
  int a;
};
class B: public A
{ public:
  B(int a, int b) : A(a),b(b) { cout << b << " B构造函数被调用\n"; }
  B(const B &p) : A(p),b(p.b)
  { cout << b << " B复制构造函数被调用\n"; }
  ~B() { cout << b << " B析构函数被调用\n"; }
protected:
  int b;
};

```

22

```

class C : public B
{ public:
  C(int a, int b, int c) : B(a,b), c(c)
  { cout << c << " C构造函数被调用\n"; }
  C(const C &p) : B(p),c(p.c)
  { cout << c << " C复制构造函数被调用\n"; }
  ~C() { cout << c << " C析构函数被调用\n"; }
protected:
  int c;
};
class D : public C
{ public:
  D(int a, int b, int c, int d) : C(a,b,c),d(d)
  { cout << d << " D构造函数被调用\n"; }
  D(const D &p) : C(p),d(p.d)
  { cout << d << " D复制构造函数被调用\n"; }
  ~D() { cout << d << " D析构函数被调用\n"; }
  void Display();
protected:
  int d;
};

```

23

```

void D::Display()
{ cout << a << " "<<b<<" "<<c<<" "<<d<<endl; }
void main()
{ D d(1,2,3,4);
  D d2(d);
  d.Display();
  d2.Display();
}

```

程序运行结果：

```

1 A构造函数被调用
2 B构造函数被调用
3 C构造函数被调用
4 D构造函数被调用
1 A复制构造函数被调用
2 B复制构造函数被调用
3 C复制构造函数被调用
4 D复制构造函数被调用
4 D析构函数被调用
3 C析构函数被调用
2 B析构函数被调用
1 A析构函数被调用
4 D析构函数被调用
3 C析构函数被调用
2 B析构函数被调用
1 A析构函数被调用
请按任意键继续...

```

24

如果把类D的构造函数和复制构造函数改为:

```
D(int a, int b, int c, int d) : a(a), b(b), c(c), d(d)
{ cout << d << " D构造函数被调用\n"; }
D(const D &p) : a(p.a), b(p.b), c(p.c), d(p.d)
{ cout << d << " D复制构造函数被调用\n"; }
```

编译结果是:

```
1>c:\test.cpp(31) : error C2512: "C": 没有合适的默认构造函数可用
1>c:\test.cpp(31) : error C2614: "D": 非法的成员初始化:"c"不是基或成员
1>c:\test.cpp(31) : error C2614: "D": 非法的成员初始化:"b"不是基或成员
1>c:\test.cpp(31) : error C2614: "D": 非法的成员初始化:"a"不是基或成员
1>c:\test.cpp(32) : error C2512: "C": 没有合适的默认构造函数可用
1>c:\test.cpp(32) : error C2614: "D": 非法的成员初始化:"c"不是基或成员
1>c:\test.cpp(32) : error C2614: "D": 非法的成员初始化:"b"不是基或成员
1>c:\test.cpp(32) : error C2614: "D": 非法的成员初始化:"a"不是基或成员
```

25

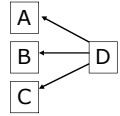
5.12 多继承 续1

2. 一个派生类同时从几个基类派生而来

```
class D : public A, protected B, private C
```

```
{ ..... }
```

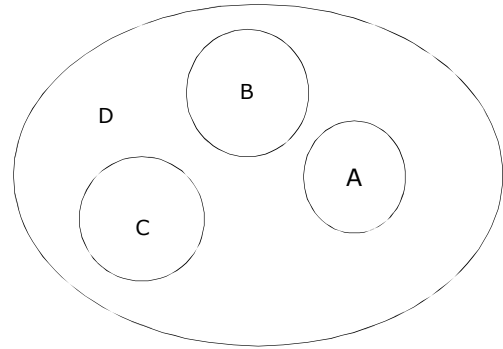
- ◆ 派生类将继承所有基类的全部成员和成员函数，并可以加入新的成员和成员函数。
- ◆ 当建立派生类的对象时，基类构造函数被依次执行，派生类的对象中所包含的基类子对象被基类的构造函数初始化。
- ◆ 但为了使用基类的某个特定的构造函数初始化派生类对象中所包含的基类子对象，必须在派生类的构造函数的成员初始化列表中显式表示对这个基类构造函数的调用。



26

```
class A
{ public: A(...) {...} // 构造函数
        A(const A &p){...} // 复制构造函数
        .....
};
class B
{ public: B(...) {...} // 构造函数
        B(const B &p){...} // 复制构造函数
        .....
};
class C
{ public: C(...) {...} // 构造函数
        C(const C &p){...} // 复制构造函数
        .....
};
class D : public A, public B, public C // 多继承
{ public: D(...) : A(...), B(...), C(...) {...} // 构造函数
        D(const D &p) : A(p), B(p), C(p){...} // 复制构造函数
        .....
};
```

27



28

执行顺序:

- 派生类D的构造函数首先使用A、B、C的构造函数依次构造出A、B、C基类对象，再构造D的成员，最终构造出整个D对象。
- 派生类D的复制构造函数首先使用A、B、C的复制构造函数依次复制构造出A、B、C基类对象，再复制构造D的成员，最终复制构造出整个D对象。基类对象的构造顺序是由继承顺序决定的。
- 派生类D的析构函数析构时，首先析构D的成员，再依次调用C、B、A的析构函数，依次析构掉C、B、A的对象成员，至此整个D对象被析构完毕。基类对象的析构顺序是由继承顺序的逆序决定的。

29

同时从几个基类派生的示例:

```
#include <iostream>
using namespace std;
class A
{ public:
    A(int x) { a=x; cout << a << " A构造函数被调用\n"; }
    A(const A &p): a(p.a)
    { cout << a << " A复制构造函数被调用\n"; }
    ~A() { cout << a << " A析构函数被调用\n"; }
protected:
    int a;
};
class B
{ public:
    B(int b) : b(b) { cout << b << " B构造函数被调用\n"; }
    B(const B &p) : b(p.b)
    { cout << b << " B复制构造函数被调用\n"; }
    ~B() { cout << b << " B析构函数被调用\n"; }
protected:
    int b;
};
```

30

```

class C
{ public:
    C(int c) : c(c) { cout << c << " C构造函数被调用\n"; }
    C(const C &p) : c(p.c)
    { cout << c << " C复制构造函数被调用\n"; }
    ~C() { cout << c << " C析构函数被调用\n"; }
protected:
    int c;
};
class D : public A, public B, public C
{ public:
    D(int a, int b, int c, int d) : A(a), B(b), C(c), d(d)
    { cout << d << " D构造函数被调用\n"; }
    D(const D &p) : A(p), B(p), C(p), d(p.d)
    { cout << d << " D复制构造函数被调用\n"; }
    ~D() { cout << d << " D析构函数被调用\n"; }
    void Display() const;
protected:
    int d;
};

```

31

```

void D::Display() const
{ cout << a << " "<<b<<" "<<c<<" "<<d<<endl; }
void main()
{ D d(1,2,3,4);
  D d2(d);
  d.Display();
  d2.Display();
}

```

程序运行结果:

```

1 A构造函数被调用
2 B构造函数被调用
3 C构造函数被调用
4 D构造函数被调用
1 A复制构造函数被调用
2 B复制构造函数被调用
3 C复制构造函数被调用
4 D复制构造函数被调用
1 2 3 4
1 D析构函数被调用
2 C析构函数被调用
3 B析构函数被调用
4 A析构函数被调用
1 A析构函数被调用
2 B析构函数被调用
3 C析构函数被调用
4 D析构函数被调用

```

而且即使你把D的构造函数和复制构造函数的构造序列修改了顺序:

```

D(int a, int b, int c, int d) : C(c), B(b), A(a), d(d)
D(D &p) : C(p), B(p), A(p), d(p.d)

```

运行时仍按照A、B、C的顺序构造。因为基类的构造顺序是由继承顺序决定的:

```

class D : public A, public B, public C

```

请按任意键继续...

32

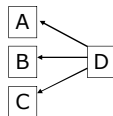
5.13 多继承的二义性

1. 一个派生类同时从几个基类派生而来的，如果几个基类中有同名的成员或成员函数。

```

class D : public A, public B, public C
{ ..... }

```



例如下面的例子中，几个基类中都有同名的保护成员a，因此D的成员函数Display()虽然可以直接访问每个基类的保护成员a，但如果写成:

```

void D::Display() const
{ cout << a << " "<<a<<" "<<a<<" "<<b<<endl; }

```

将出现二义性(ambiguous)，因为编译系统无法确定是要打印哪个基类的保护成员a，因而显示编译错误，需要在相应成员或成员函数前加基类名限定(像A::)。

33

```

#include <iostream>
using namespace std;
class A
{ public: A(int x) { a=x; } // 构造函数
  A(const A &p): a(p.a){ } // 复制构造函数
protected: int a;
};
class B
{ public: B(int x): a(x) { } // 构造函数
  B(const B &p): a(p.a){ } // 复制构造函数
protected: int a;
};
class C
{ public: C(int x) { a=x; } // 构造函数
  C(const C &p) { a=p.a; } // 复制构造函数
protected: int a;
};
class D : public A, public B, public C
{ public: D(int a, int b, int c, int d): A(a), B(b), C(c)
  { b=d; } // 构造函数
  D(const D &p) : A(p), B(p), C(p){b=p.b; } //复制构造函数
  void Display() const;
protected: int b;
};

```

34

```

void D::Display() const
{ cout << a << " "<<a<<" "<<a<<" "<<b<<endl; } // line 26
void main()
{ D d(1,2,3,4);
  D d2(d);
  d.Display();
  d2.Display();
}

```

35

编译结果:

```

1> test.cpp(26) : error C2385: 对“a”的访问不明确
1>     可能是“a”(位于基“A”中)
1>     也可能是“a”(位于基“B”中)
1>     也可能是“a”(位于基“C”中)
1> test.cpp(26) : error C2385: 对“a”的访问不明确
1>     可能是“a”(位于基“A”中)
1>     也可能是“a”(位于基“B”中)
1>     也可能是“a”(位于基“C”中)
1> test.cpp(26) : error C2385: 对“a”的访问不明确
1>     可能是“a”(位于基“A”中)
1>     也可能是“a”(位于基“B”中)
1>     也可能是“a”(位于基“C”中)
1> test - 3 个错误, 0 个警告

```

36

而如果要改为:

```
void D::Display( ) const
{ cout << A::a << " "<< B::a << " "<< C::a << " "<<b<<endl; }
void main( )
{ D d(1,2,3,4); D d2(d); d.Display( ); d2.Display( ); }
编译无任何错误! 运行结果:
1 2 3 -858993460
1 2 3 -858993460
请按任意键继续...
```

结果为什么是这样呢? 因为D的构造函数:

```
D(int a, int b, int c, int d): A(a), B(b), C(c) {b=d;}
```

中参数b与成员b同名, b=d; 没有将d的值赋给成员b, 应改为:

```
D(int a, int b, int c, int d): A(a), B(b), C(c) {this->b=d; }
```

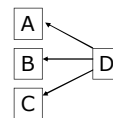
修改后运行结果:

```
1 2 3 4
1 2 3 4
请按任意键继续...
```

37

2. 一个派生类同时从几个基类派生而来的, 如果派生类和几个基类中有同名的成员或成员函数。

```
class D : public A, public B, public C
{ ..... }
```



例如下面的例子中, 派生类和几个基类中都有同名的保护成员a, 因此D的成员函数Display()虽然可以直接访问每个基类的保护成员a, 但如果写成:

```
void D::Display( ) const
{ cout << a<< " "<<a<< " "<<a<< " "<<a<<endl; }
```

38

```
#include <iostream>
using namespace std;
class A
{ public: A(int x) { a=x; } // 构造函数
  A(const A &p): a(p.a) { } // 复制构造函数
  protected: int a;
};
class B
{ public: B(int x): a(x) { } // 构造函数
  B(const B &p): a(p.a) { } // 复制构造函数
  protected: int a;
};
class C
{ public: C(int x) { a=x; } // 构造函数
  C(const C &p) { a=p.a; } // 复制构造函数
  protected: int a;
};
class D : public A, public B, public C
{ public: D(int a, int b, int c, int d): A(a), B(b), C(c)
  { this->a=d; } // 构造函数
  D(const D &p): A(p), B(p), C(p) { a=p.a; } // 复制构造函数
  void Display( ) const;
  protected: int a;
};
```

39

```
void D::Display( ) const
{ cout << a<< " "<<a<< " "<<a<< " "<<a<<endl; } // line 26
void main( )
{ D d(1,2,3,4); D d2(d); d.Display( ); d2.Display( ); }
编译无任何错误! 运行结果:
4 4 4 4
4 4 4 4
请按任意键继续...
```

这是为什么呢? 因为派生类D的同名成员a掩蔽了基类的同名成员, D::Display()直接看到的a是派生类D的a, 如果想打印每个基类的成员a, 必须在相应成员或成员函数前加基类名限定:

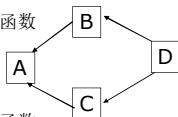
```
void D::Display( ) const
{ cout << A::a << " "<< B::a << " "<< C::a << " "<<a<<endl; }
修改后运行结果:
1 2 3 4
1 2 3 4
请按任意键继续...
```

40

3. 一个派生类同时从几个基类派生而来的, 如果几个基类又是从同一个更老的共同基类派生而来。

例如: B, C都是从A派生来的, 而D又是从B、C派生来的:

```
class A
{ ..... };
class B : public A
{ public: B(...): A(...) { ... } // 构造函数
  B(const B &p): A(p) { ... } // 复制构造函数
  .....
};
class C : public A
{ public: C(...): A(...) { ... } // 构造函数
  C(const C &p): A(p) { ... } // 复制构造函数
  .....
};
class D : public B, public C
{ public: D(...): B(...), C(...) { ... } // 构造函数
  D(const D &p): B(p), C(p) { ... } // 复制构造函数
  .....
};
```



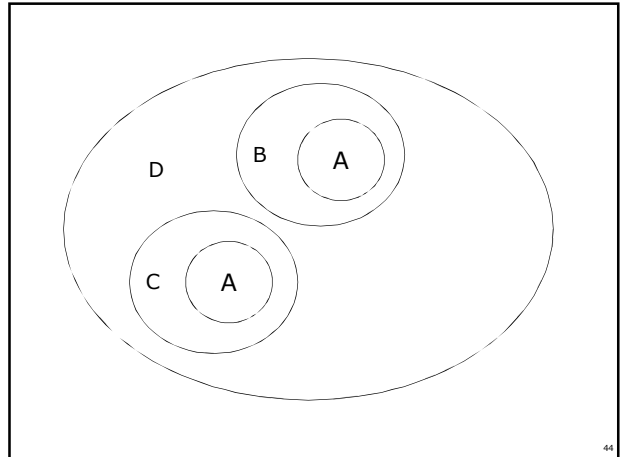
41

```
#include <iostream>
using namespace std;
class A
{ public: A(int x) { a=x; } // 构造函数
  A(const A &p): a(p.a) { } // 复制构造函数
  protected: int a;
};
class B : public A
{ public: B(int x, int y): A(x) { b=y; } // 构造函数
  B(const B &p): A(p), b(p.b) { } // 复制构造函数
  protected: int b;
};
class C : public A
{ public: C(int x, int y): A(x) { c=y; } // 构造函数
  C(const C &p): A(p) { c=p.c; } // 复制构造函数
  protected: int c;
};
class D : public B, public C
{ public: D(int a, int b, int c, int d): B(a,b), C(a, c)
  { this->d=d; } // 构造函数
  D(const D &p): B(p), C(p) { d=p.d; } // 复制构造函数
  void Display( ) const;
  protected: int d;
};
```

42

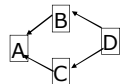
```
void D::Display( ) const
{ cout << a<<" "<<b<<" "<<c<<" "<<d<<endl; } // line 26
void main( )
{ D d(1,2,3,4); D d2(d); d.Display( ); d2.Display( ); }
编译结果:
test.cpp(26) : error C2385: 对“a”的访问不明确
1> 可能是“a”(位于基“A”中)
1> 也可能是“a”(位于基“A”中)
1>test - 1 个错误, 0个警告
出现错误的原因, 是编译器无法确定D::Display( )要访问B和C中哪一个的
a, 因为B和C中都有基类A, a是重复存在的。如果想打印其中一个基
类的成员a, 必须在相应成员或成员函数前加基类名限定:
void D::Display( ) const
{ cout << B::a<<" "<<b<<" "<<c<<" "<<d<<endl; } // line 26
修改后编译没有错误! 运行结果:
1 2 3 4
1 2 3 4
请按任意键继续...
```

43



44

5.14 虚基类



- 一个派生类同时从几个基类派生而来的, 如果几个基类又是从同一个更老的共同基类派生而来。
- 前面的例如: B, C都是从A派生来的, 而D又是从B、C派生来的, 这样D中将有2个A基类的副本, 为了访问A基类的成员或成员函数, 需要加限定名B::或C::。
- 虽然D中有2个A基类的副本有时候是必要的, 但在许多情况下也是人们不希望出现的, 因为多个基类的副本同时存在会造成占用更多内存, 访问时不但麻烦还容易出错。
- 为了解决这个问题, C++引入了虚基类(virtual base class), 虚基类将使得派生类D在继承间接共同基类A时只保留一个A的副本。

45

```
#include <iostream>
using namespace std;
class A
{ public: A(int x) { a=x; } // 构造函数
  A(const A &p): a(p.a){ } // 复制构造函数
  protected: int a;
};
class B : virtual public A
{ public: B(int x, int y): A(x) {b=y; } // 构造函数
  B(const B &p): A(p), b(p.b){ } // 复制构造函数
  protected: int b;
};
class C : virtual public A
{ public: C(int x, int y): A(x) {c=y; } // 构造函数
  C(const C &p): A(p) {c=p.c; } // 复制构造函数
  protected: int c;
};
class D : public B, public C
{ public: D(int a, int b, int c, int d): B(a,b), C(a, c)
  {this->d=d; } // 构造函数 // line 20
  D(const D &p): B(p), C(p) {d=p.d; } //复制构造函数//line 21
  void Display( ) const;
  protected: int d;
};
```

46

```
void D::Display( ) const
{ cout << a<<" "<<b<<" "<<c<<" "<<d<<endl; } // line 26
void main( )
{ D d(1,2,3,4); D d2(d); d.Display( ); d2.Display( ); }
```

编译结果:
1>test.cpp(20) : error C2512: “A::A”: 没有合适的默认构造函数可用
1>test.cpp(21) : error C2512: “A::A”: 没有合适的默认构造函数可用
1>test - 2 个错误, 0个警告

出现错误的原因是: 当把B和C继承A的方式改为virtual继承方式后, A在最终的派生类D中只有一个副本, 这样还间接通过B和C的构造函数和复制构造函数初始化基类A的成员, 可能会因为B和C各自给不同的值初始化A, 造成数据的不一致性。

因此, 此时将由D的构造函数和复制构造函数直接调用A的构造函数和复制构造函数进行初始化, 即使D的构造列表中并没有给出A, 系统也会按默认方式调用缺省值的A构造函数, 但本程序中并没有缺省值的A构造函数, 因此出现编译错误。

程序应该改为:

47

```
#include <iostream>
using namespace std;
class A
{ public: A(int x) { a=x; } // 构造函数
  A(const A &p): a(p.a){ } // 复制构造函数
  protected: int a;
};
class B : virtual public A
{ public: B(int x, int y): A(x) {b=y; } // 构造函数
  B(const B &p): A(p), b(p.b){ } // 复制构造函数
  protected: int b;
};
class C : virtual public A
{ public: C(int x, int y): A(x) {c=y; } // 构造函数
  C(const C &p): A(p) {c=p.c; } // 复制构造函数
  protected: int c;
};
class D : public B, public C
{ public: D(int a, int b, int c, int d): A(a), B(5, b), C(6, c)
  {this->d=d; } // 构造函数 // line 20
  D(const D &p): A(p), B(p), C(p) {d=p.d; } //复制构造函数
  void Display( ) const;
  protected: int d;
};
```

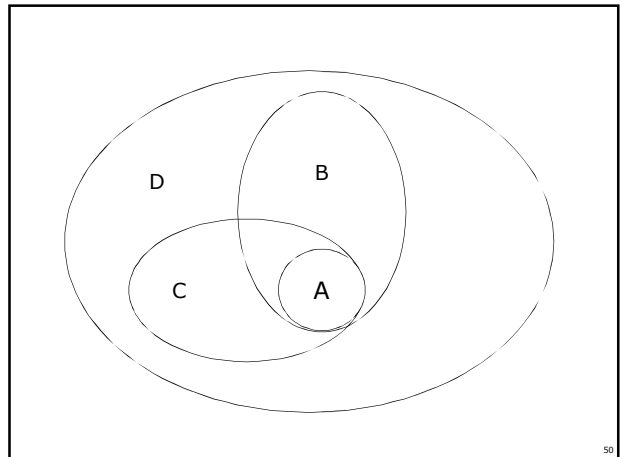
48


```
void D::Display( ) const
{ cout << a<<" "<<b<<" "<<c<<" "<<d<<endl; }
void main( )
{ D d(1,2,3,4); D d2(d); d.Display( ); d2.Display( ); }
修改后编译没有错误！运行结果：
1 2 3 4
1 2 3 4
请按任意键继续...
```

注意：1. 虽然D的构造函数和复制构造函数直接调用A的构造函数和复制构造函数进行初始化，但在书写B和C的构造函数和复制构造函数时，仍需调用A的构造函数和复制构造函数进行初始化，因为B和C很可能不止是D使用，B和C可以单独直接实例化生成对象，此时B和C的构造函数和复制构造函数将直接通过调用A的构造函数和复制构造函数来生成基类A的成员a并初始化。

2. 在生成D的对象d时，其中基类A的成员a只有一个副本，说明A的构造函数和复制构造函数只会被使用过一次，而不是3次，这个是通过virtual实现的。因为程序中故意给的B(5, b), C(6, c) 的值5、6并没有赋给a，说明此时B和C的构造函数并没有真去调用A的构造函数。

49



50

第8次作业：

见网络学堂第8次作业

2题必做，1题选做

51